

4D Sessions: From Desktop to Web in a Single Shared Session

By Al Mahdi Bakkali, Technical Support Engineer, 4D Inc.

Technical Note 26-06

Table of Contents

Table of Contents	2
Abstract.....	3
Introduction	3
System Requirements	4
Why Client-Side Sessions?	4
Reading Session Metadata on the Client.....	5
Session.info object	5
Properties Reference	6
Session.createOTP() for Desktop Sessions	7
Sharing a Session Between Desktop and Web	8
The OTP URL Mechanism	8
Mobile Device Pairing with a Second OTP	9
Session.storage - Sharing state across devices.....	9
Changes in Session functions.....	10
The Complete Application Flow.....	12
Best Practices & Security Considerations.....	14
OTP Lifetime	14
HTTPS in Production	14
Privilege Cleanup at Case Closure	14
Troubleshooting Common Issues	14
Session.storage.data is Null in the Web Handler.....	14
OTP-Linked URL Shows "Session Expired"	15
Conclusion	15

Abstract

The 4D Secure OTP demonstration application is a cross-platform identity and document verification system. Designed to eliminate redundant session management architecture, the application leverages client-side Session objects to surface session metadata and one-time-password generation directly from 4D remote desktop clients without requiring server-side delegation.

Session state is synchronized across three concurrent access points: a 4D desktop form, a web console, and a mobile device. The system implements four interconnected session capabilities introduced across 4D 21 and extended in 4D 21 R3: direct **Session.info** property access from remote clients, desktop-scoped **Session.createOTP()** invocation, a cross-platform session-sharing mechanism that binds different clients to a single session context, and a **Session.storage** shared object that propagates state changes across all active participants in real time.

Introduction

In modern 4D applications, session management sits at the intersection of data access, user experience, and architecture. A session is the runtime context that determines what a user can do, what data they can see, and how different parts of the application can communicate. In a multi-channel application where a desktop user, a web interface, and a mobile device all need to cooperate on the same task, session management becomes the critical infrastructure that makes this coordination possible.

Prior to 4D 21 R3, remote user sessions already existed and could be accessed on the server, but the Session command returned `Null` when executed directly from a 4D remote client. Session information was therefore only available in server-side execution context, such as HTTP handlers, REST handlers, stored procedures, or methods explicitly dispatched to the server. As a result, developers who needed session data in client-side code, such as form methods or object methods, often had to create dedicated server-side methods whose sole purpose was to relay session information back to the client, adding unnecessary complexity.

The 4D 21 R3 extension of the Session command to remote clients significantly reduces this layer of complexity. A form method can now call **Session.info** or access properties such as **Session.userName** directly from client-side code, and it can invoke methods such as **Session.createOTP()** to generate a one-time password that allows a web browser to join the same user session. Combined with the session-sharing capabilities of the One-Time Password mechanism, these features enable sophisticated multi-channel workflows that previously required additional infrastructure and custom server-side methods.

This document uses a practical demonstration application, 4D Secure OTP, to illustrate these capabilities in context. 4D Secure OTP is an identity verification console for KYC (Know Your Customer) workflows: a compliance officer opens a verification session on their desktop, a subject proves their presence from a mobile device through a QR-code challenge, uploads an identity document, and the officer reviews the submission and makes a decision; all within a single shared 4D session. This scenario exercises many of the session-management capabilities now available directly from remote client code.

System Requirements

The demonstration 4D application accompanying this technical note was developed for both Windows and macOS and requires the following minimum environment.

- **4D 21 R3** - required for **Session** command availability from 4D remote clients and Session.info property
- **4D 21 or later** - required for **Session.createOTP()** on desktop sessions; Boolean return value from **setPrivileges()** / **clearPrivileges()**
- **4D Server license** with Web Server enabled — required to serve HTTP routes and host the user portal
- **4D Remote license** — required for the user desktop session; this is the session that initiates the OTP
- **Windows 11 or macOS Tahoe**

Note: Desktop sessions are available in three configurations: remote user sessions in client/server applications (the session object is accessible on both the server and the client); stored procedure sessions (the virtual server session shared by all stored procedures); and standalone sessions in single-user 4D. **Standalone sessions are particularly useful during development and can be used for this application.** Users can use the full session functions, including OTP generation and web access sharing, without a server setup, regardless of whether the final application targets single-user or client/server deployment.

Why Client-Side Sessions?

Before 4D 21 R3, client-side code that needed access to session information had to obtain it through server-side execution. Client-side code could not directly access the **Session** object and therefore depended on server-side intermediary methods to retrieve session information.

With 4D 21 R3, calling Session from a 4D remote client returns the current remote user session object directly. There are no additional server-side requests required simply to read session information.

The Session command now exposes the following capabilities directly to client-side code:

- **Session.info** - session metadata including username, machine name, IP address, session identifier, session type, and creation timestamp
- **Session.id** - direct access to commonly used session properties
- **Session.storage** - session-scoped storage shared by the session
- **Session.createOTP()** - generate a one-time password that allows a web client to join the same session
- **Session.hasPrivilege()** / **Session.getPrivileges()** / **Session.isGuest()** - privilege inspection functions available on both client and server

Some operations remain server-side only:

- **Session.setPrivileges()**
- **Session.clearPrivileges()**

Attempting to call these methods directly from a 4D remote client generates an error because privilege modification remains a server's responsibility.

Together, these capabilities enable multi-channel applications in which a desktop client and one or more web clients can collaborate within a single shared session context without requiring custom infrastructure.

Reading Session Metadata on the Client

Session.info object

The Session.info property returns an object containing identity and context information about the current session. In the context of a 4D remote client, it provides the metadata that identifies the user: the user identity, connection context, session creation timestamp, and other properties explored in the next section.

In the demonstration application, the Session_Form desktop form uses Session.info on the On Load event to populate the user's identity panel. This is the first form the user sees, and it demonstrates that session metadata is available immediately in the client context, without any method dispatched to the server.

Session information
Session.info : live snapshot

● Active

Type	standalone
User name	Designer
Machine name	ZBF-0599
IP address	localhost
Session ID	4DDD934F...B8DA3A
Created	2026-06-25 01:01
State	Active

Following is an extract of the form method for Session_Form. All values are derived directly from Session.info with no server execution:

```
Case of  
: (Form event code=On Load)
```

```

var $info : Object:=Session.info

OBJECT SET TITLE(*; "valType"; String($info.type))
OBJECT SET TITLE(*; "valUser"; String($info.userName))
OBJECT SET TITLE(*; "valMachine"; String($info.machineName))
OBJECT SET TITLE(*; "valIP"; String($info.IPAddress))
// continue populating form objects
End case

```

Properties Reference

The following example properties are available from Session.info for a remote (desktop) session. All properties are read-only.

Property	Type	Description
type	Text	Session type. Returns "remote" for a 4D remote client connection.
userName	Text	The 4D user name from the user directory, as entered during application login.
machineName	Text	The network host name of the client machine.
IPAddress	Text	The IP address and port of the client connection, formatted as address:port.
ID	Text	The unique session identifier (UUID format). Stable for the lifetime of the session.
creationDateTime	Text	ISO 8601 timestamp of when the session was established, e.g. 2026-06-24T09:31:00.000Z.

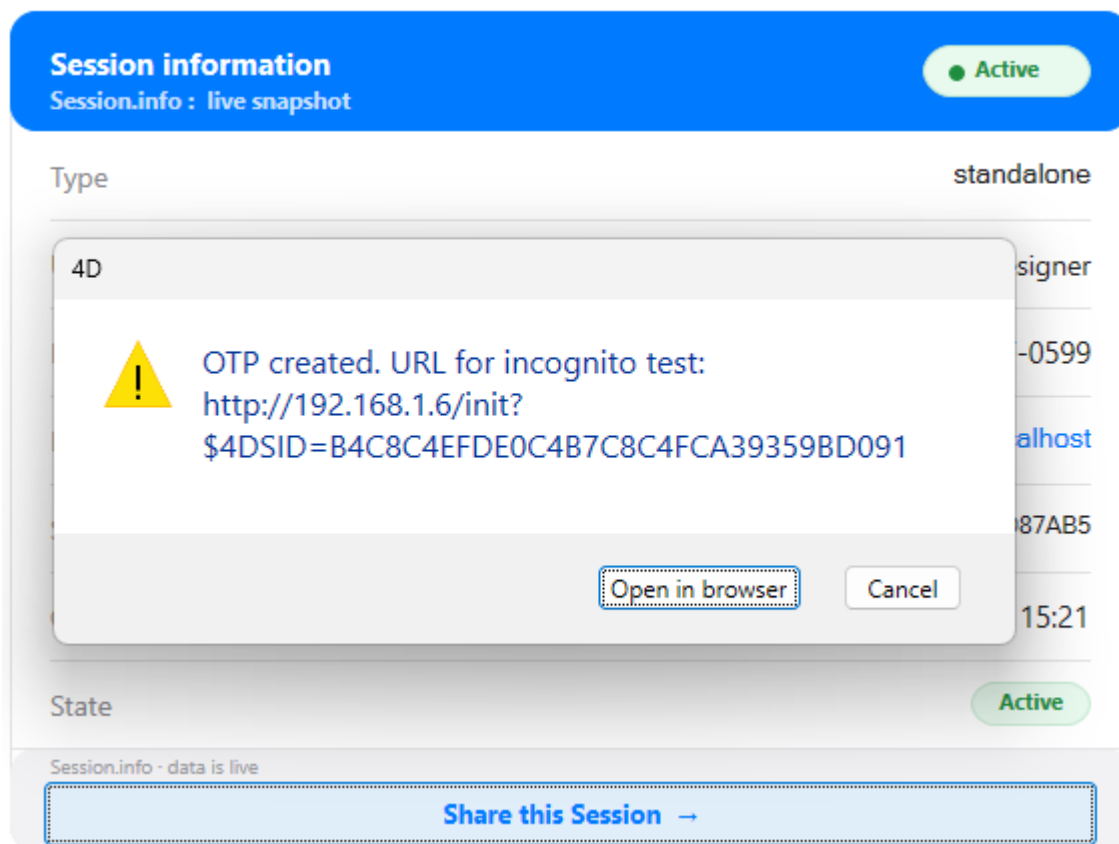
Availability: Session.info is available in both remote client contexts (4D 21 R3) and in server-side execution contexts. The properties returned differ by session type: a web session exposes userName but not machineName while a remote session exposes the full set above and more.

See the [4D documentation](#) for the complete property matrix by session type.

Session.createOTP() for Desktop Sessions

Session.createOTP() was originally introduced for web sessions, enabling a web client to share its session with another web client by embedding the generated token in a URL. With 4D 21, this method was extended to work with remote user sessions (desktop sessions), making it possible to invite a web browser into an existing desktop session.

The method generates a short-lived, single-use token that, when appended to a URL as the \$4DSID query parameter, instructs the 4D server to associate the new HTTP connection with the session that created the token.



Here is an extract of the method in question:

```
var $token : Text:=Session.createOTP(10)
```

The token expires after the system-configured OTP timeout; after that, navigating to the URL return a message of expired session.

In the demonstration application, the getOTP project method creates the token and initializes the shared session storage that could only be shared with a valid OTP. Here is an extract of the code:

```
Use (Session.storage)
    Session.storage.desktopMessage:=New shared object("openedAt"; Timestamp)
End use
```

The button object method in `Session_Form` receives this token, constructs the full URL, and opens it in the system browser. The resulting URL carries the OTP as the `$4DSID` query parameter, which is the standard 4D mechanism for session-linked URLs. Here in an extract of the code:

```
var $otp : Text:=getOTP
var $port : Integer:=$serverInfo.options.webPortID
var $host : Text:=$serverInfo.options.webIPAddressToListen[0]
var $url : Text

$url:="http://" + $host + "/init?$4DSID=" + $otp

OPEN URL($url)
```

The token attached to the URL will match the generated OTP and therefore grant the session sharing to the browser. This code will open an URL with the following structure in the browser.

192.168.1.6/init?\$4DSID=B4C8C4EFDE0C4B7C8C4FCA39359BD091

Sharing a Session Between Desktop and Web

The OTP URL Mechanism

The OTP mechanism is the architectural centerpiece of the 4D Secure OTP application. The user is running 4D as a remote client. The web console runs in a browser. The subject's identity verification happens on a mobile device. These are three separate client types using three different protocols, yet they all participate in a single, shared session because each access point was admitted via an OTP generated from that session.

When the user clicks **Share session** in the **Session_Form**, the following sequence occurs:

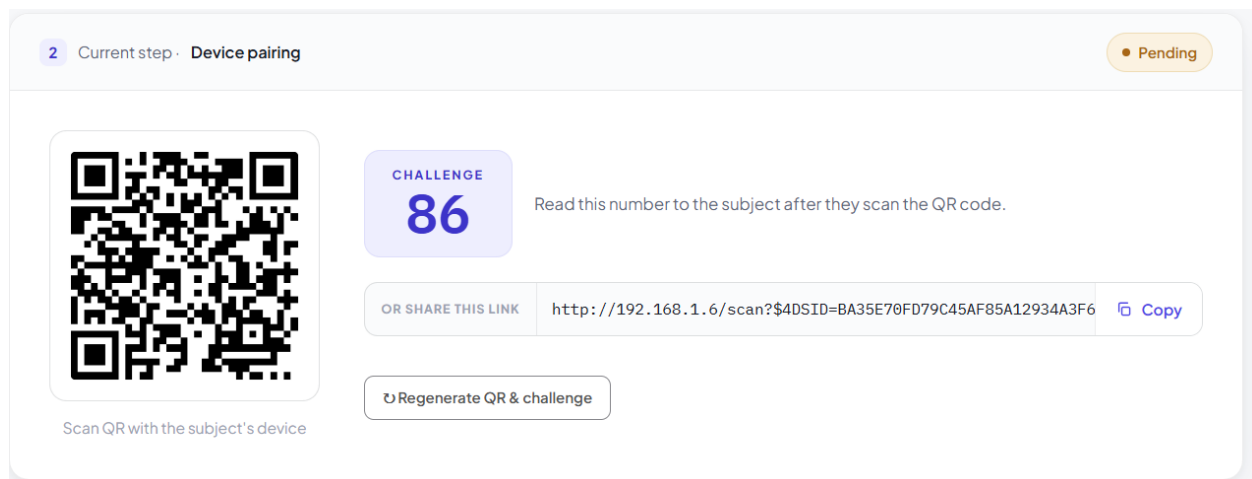
1. **getOTP** runs on the server. It initializes `Session.storage` and returns a token.
2. The method constructs the URL: **`http://<server>/init?$4DSID=<token>`**
3. The browser navigates to this URL. 4D matches the token and links the HTTP session to the desktop session.
4. All subsequent requests from this browser window run in the context of the user's desktop session.

This is the standard 4D session-sharing URL pattern. The **\$4DSID** parameter is consumed on the first request. The server then sets a session cookie in the browser response, and subsequent requests use that cookie rather than the token. The token itself is single-use and immediately invalidated once consumed.

Mobile Device Pairing with a Second OTP

The user's mobile device joins the session through a separate OTP generated within the web context. When the user navigates to the /pair route, the HTTP handler generates a second token, encodes it into a QR code URL, and renders the pairing screen:

```
var $token : Text:=Session.createOTP(5)
var $host : Text:=$request.getHeader("host")
var $scanURL : Text:="http://" + $host + "/scan?$4DSID="+$token
var $qrURL : Text:="https://api.qrserver.com/v1/create-qr-code/?size=240x240&data="+$scanURL
```

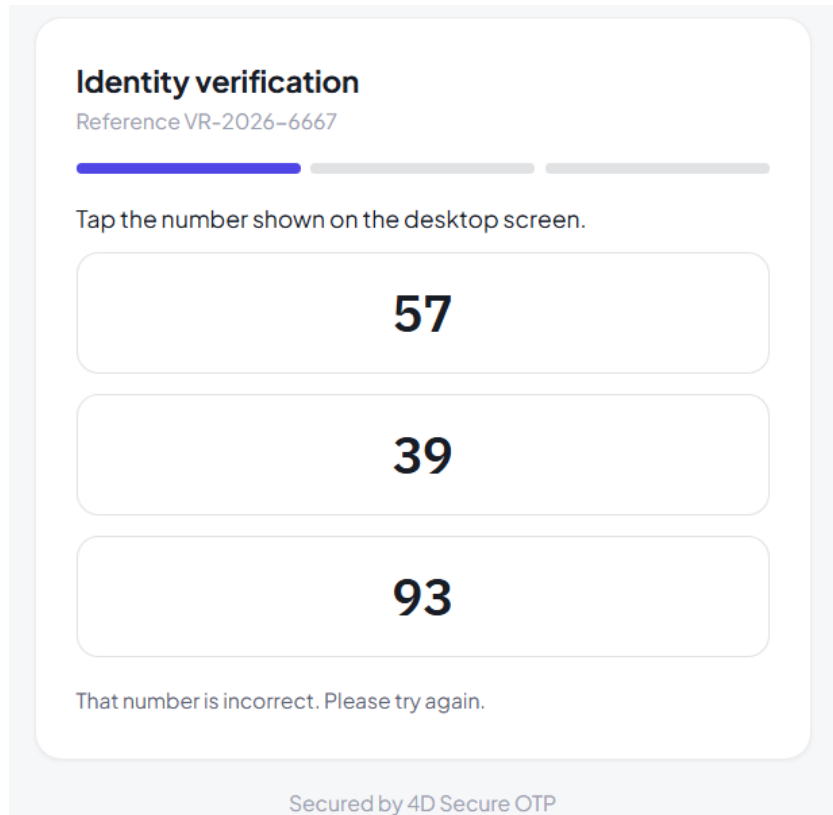


When the subject scans the QR code with their mobile device and their browser navigates to the encoded URL, the 4D server admits them into the same session. At this point, three distinct clients are sharing one session: the 4D remote desktop client, the user's browser window, and the user's mobile browser.

Session.storage - Sharing state across devices

Session.storage is a shared object that persists for the lifetime of the session. Because all three clients share the same session, any write to **Session.storage** is immediately visible to all of them on their next read. Based on this feature, the challenge-response verification option, among others, is implemented. In the example of the challenge-response, the initial challenge is shared through **Session.storage** and later verification also check the selected number against this shared property.

The following screenshot shows and incorrect selection by the user.



Changes in Session functions

Since 4D 21, both **Session.setPrivileges()** and **Session.clearPrivileges()** return a boolean value indicating whether the operation succeeded. Previously these methods returned nothing, leaving developers with no direct way to confirm that the privilege change took effect. The boolean return enables tighter control flow in security-sensitive code paths.

In the demonstration project, the privileges, configured in **roles.json** control access to the **uploadPhoto** class function. The user must successfully answer the challenge before the privilege to **/fileUpload** is granted.

The image shows a user verification interface. At the top, a green rounded rectangle contains the text '✓ Identity confirmed'. Below this, the text 'Upload your verification photo to complete the process.' is displayed. A progress bar with two blue segments and one grey segment is shown. Underneath the progress bar is a dashed rounded rectangle containing a 'Choose File' button and the text 'No file chosen'. Below this is a large blue rounded rectangle with the text 'Upload verification photo'. At the bottom, the text 'Secured by 4D Secure OTP' is displayed.

The **/reply** handler grants the **verified_user** privilege upon a correct response:

```
If ($correct)
  Use (Session.storage.data)
    Session.storage.data.status:="verified"
    Session.storage.data.message:="Identity confirmed. Please upload your photo."
  End use
  If (Session.setPrivileges("verified_user"))
    //continue
  end if
```

The upload handler uses **Session.hasPrivilege()** to verify the privilege before processing the file. Here is an extract of the code:

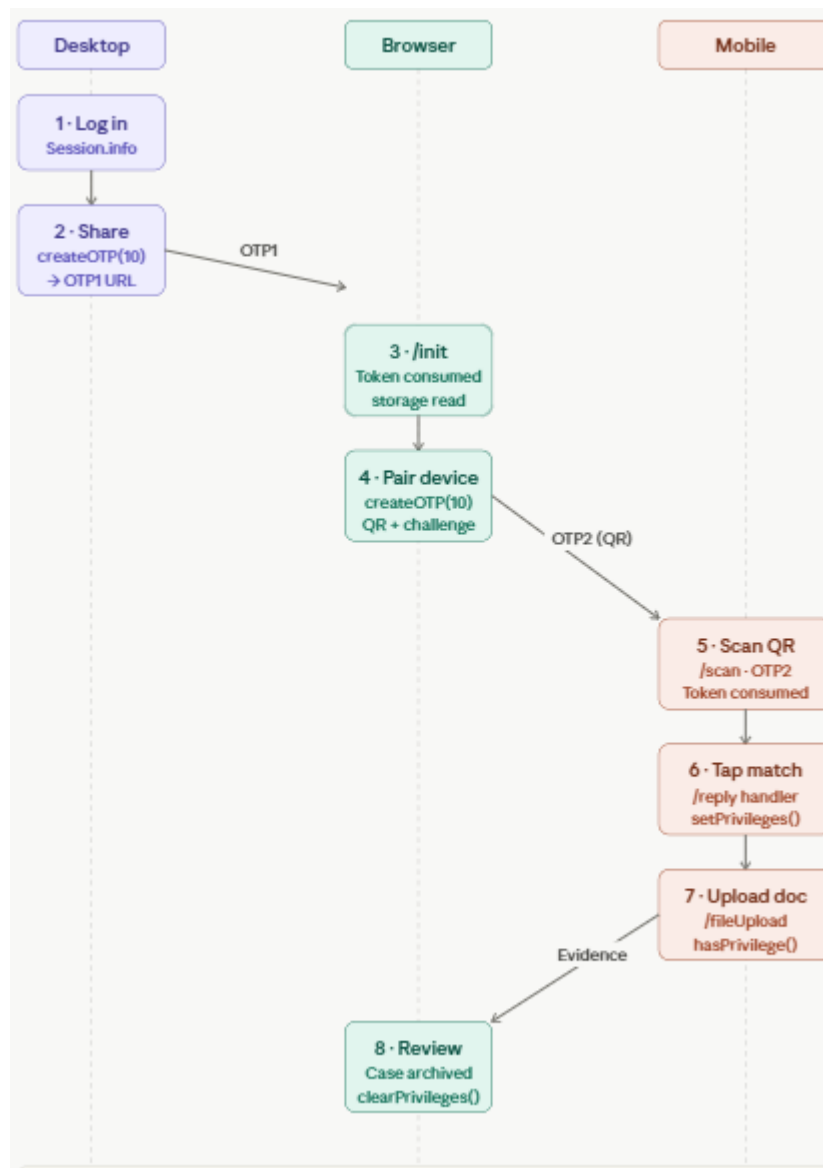
```
Function upload($request : 4D.IncomingMessage) : 4D.OutgoingMessage
  If (Not(Session.hasPrivilege("verified_user")))
    return This.jsonResult(New object("ok"; False; "message"; \
      "Access denied. Verify the challenge first."); 403)
  End if
  // ... proceed with upload ...
```

When the user records a terminal decision (approved or rejected), **clearPrivileges()** revokes all session privileges and a fresh case is opened. Using the boolean return, the application could optionally verify that the clear succeeded before continuing:

```
If ($outcome="approved") | ($outcome="rejected")
  If( Session.clearPrivileges() ) // returns Boolean since 4D 21
  //proceed
  End if
End if
```

The Complete Application Flow

The following describes how all the session capabilities described in this document combine into the complete 4D Secure OTP verification workflow.



The \$4DSID parameter is consumed on the first request: the server sets up a session cookie in the browser response, and subsequent requests use that cookie rather than the token. The token itself is single-use and immediately invalidated once consumed. **Please use private navigation as a second device to avoid retrieving the same session.**

Each submitted report is assigned a unique reference (e.g. VR-2026-6667) and linked to the originating session via a stored session identifier. Submitted evidence files are integrity-verified using SHA-256. The image is hashed at the time of submission, ensuring that its content cannot be altered or tampered with after the fact.

Case report VR-2026-6667

Generated 2026-06-25 · 15:31 UTC

Print / Save PDF

CASE DETAILS

Reference	VR-2026-6667
Channel	QR · Mobile
Created	2026-06-25 · 15:25 UTC
Submitted	2026-06-25 · 15:31 UTC
Session	64653DD775FC4D27...

EVIDENCE

File	{878DCE6B-A7A2-45A5-A4A5-B6BA289B8EC9}.png
Type	PNG
Size	73.7 KB
SHA-256	cb8027e843932557ce36fb239ba485fb2633e157b2df5910a32c9e932c0d006c

Once a document is uploaded by the user, an operator can review it and set its status to approved, rejected, or more info requested, along with a reason selected from a dropdown. Each case is accessible via its unique reference, making it easy to track submissions and act on them individually. The Reports page provides operators with a consolidated view of all submitted verification cases.

Reports

• 3 total

All verification case reports

CASES				
REFERENCE	SUBMITTED	FILE	STATUS	
VR-2026-9855	2026-06-25 · 15:34 UTC	{878DCE6B-A7A2-45A5-A4A5-B6BA289B8EC9}.png	• Info requested	Open report ↗
VR-2026-4463	2026-06-25 · 15:34 UTC	{878DCE6B-A7A2-45A5-A4A5-B6BA289B8EC9}.png	• Rejected	Open report ↗
VR-2026-6667	2026-06-25 · 15:31 UTC	{878DCE6B-A7A2-45A5-A4A5-B6BA289B8EC9}.png	• Approved	Open report ↗

Best Practices & Security Considerations

OTP Lifetime

Tokens generated by **Session.createOTP()** are single-use and expire after the system's configured OTP timeout. Always generate a fresh token immediately before presenting it to the user and never cache or reuse a previously generated token. In the demonstration application, the **/pair** route has a button that regenerates the challenge and creates a new OTP on every click, ensuring that the displayed QR code always reflects a valid, current token.

HTTPS in Production

The OTP token is transmitted in the URL query string. Anyone who intercepts the URL before it is consumed can join the session. In the demonstration application, HTTP is used for simplicity, but production deployments must use HTTPS to prevent token interception in transit. The **cert.pem** and **key.pem** files in the 4D project folder are the standard locations for the web server's TLS certificate and key.

Privilege Cleanup at Case Closure

Always call **Session.clearPrivileges()** when a verification case reaches a terminal state. Failing to do so leaves the **verified_user** privilege active into the next case, which would allow a subject to bypass the challenge-response step entirely and upload evidence without identity verification. The demonstration application clears privileges on every approved or rejected decision before opening a fresh case.

Troubleshooting Common Issues

Session.storage.data is Null in the Web Handler

If a method generating the OTP is not marked "Execute on Server", the storage initialization runs in the client, which HTTP handlers never see. Confirm that the method includes this property.

OTP-Linked URL Shows "Session Expired"

The `/init` and `/scan` routes guard against null session data and return a 403 with a "Session expired" message if it is absent. This can occur for two main reasons. If the storage initialization and OTP creation are not running on the Server. In this case, check the property "Execute on Server"

It can also happen if the OTP has expired, which is an expected behavior. For demonstration purposes, we set the OTP timeout to be 10 seconds. If this timeout is exceeded, the browser will show a 403 error. To fix this, simply regenerate the OTP thanks to the dedicated button.

NOTE: Make sure to always open the URL in another device, or in private navigation, to avoid retrieving the same session.

Conclusion

The extension of the Session command to 4D remote clients in version 21 R3 reduces server-side dependencies, including reading session metadata and generating session-sharing tokens can now both be initiated from the client.

The 4D Secure OTP demonstration application applies all four of the capabilities documented in this note: **Session.info** for identity display in a desktop form, **Session.createOTP()** for both the initial desktop-to-web sharing and the subsequent mobile device pairing. **Session.storage** is the shared real-time state across desktop, browser, and mobile. The functions **setPrivileges()** and **clearPrivileges()** ensure privileges are set correctly and can be ensured with the boolean return.